

Section 3.7 – Travaux Pratiques : Simulation et Bootstrap

HAMLIL Mohamed

2025-12-31

Table des matières

1	Section 3.7 – Travaux Pratiques	1
1.1	Vue d'ensemble	1
1.2	3.7.1 Simulation de Lancers de Dé et Estimation d'une Proportion	1
1.2.1	Problématique	1
1.2.2	Étape 1 : Fonction de Simulation	2
1.2.3	Observation	2
1.2.4	Étape 2 : Estimation du Biais et de la Variance par Monte-Carlo	3
1.2.5	Étape 3 : La Méthode du Bootstrap	5
1.3	3.7.2 Simulation de Variables Aléatoires	7
1.3.1	Contexte	7
1.3.2	Méthode 1 : Transformation Inverse pour Loi Exponentielle	7
1.3.3	Méthode 2 : Simulation Discrète Uniforme	8
1.4	Résumé et Points Clés	10
1.4.1	Ce que nous avons appris	10
1.4.2	Conseils Pratiques	10

1 Section 3.7 – Travaux Pratiques

1.1 Vue d'ensemble

Ce notebook regroupe des exercices pratiques sur :

- **La méthode du bootstrap** : estimer le biais et la variance d'un estimateur à partir d'un seul échantillon
- **La simulation de variables aléatoires** : générer des données selon des lois qui n'existent pas directement dans R

Objectifs pédagogiques :

- Comprendre le fonctionnement de la méthode du bootstrap
- Maîtriser les méthodes classiques de simulation (transformation inverse, simulation discrète)
- Comparer les estimations empiriques aux valeurs théoriques

Structure : Chaque section alterne entre théorie mathématique et implémentation pratique en R.

1.2 3.7.1 Simulation de Lancers de Dé et Estimation d'une Proportion

1.2.1 Problématique

On simule n lancers d'un dé équilibré. L'objectif est d'estimer la probabilité d'obtenir un 4 par la fréquence empirique $\hat{p}$.

Questions clés :

- Est-ce que $\hat{p}$ est un bon estimateur de $p = 1/6$?
- Comment varie $\hat{p}$ selon l'échantillon ?
- Comment estimons-nous le biais et la variance ?

1.2.2 Étape 1 : Fonction de Simulation

Créons une fonction `lancer_le_de(n)` qui simule n lancers d'un dé équilibré.

```
# ===== FONCTION: Simuler n lancers d'un dé équilibré =====

lancer_le_de <- function(n) {
  # sample() tire des valeurs dans 1:6 avec remise
  # replace=TRUE signifie "avec remise" (indépendance des lancers)
  return(sample(1:6, n, replace = TRUE))
}

# === EXEMPLE: 20 lancers ===
set.seed(10)
echantillon <- lancer_le_de(20)
cat("20 lancers du dé:\n")
```

20 lancers du dé:

```
print(echantillon)
```

```
[1] 3 1 2 4 6 3 2 2 2 5 6 6 3 6 2 5 5 5 1 4
```

```
# Estimer la probabilité d'obtenir un 4 dans cet échantillon
p_empirique <- sum(echantillon == 4) / length(echantillon)
cat("\nProbabilité empirique de 4:", p_empirique, "\n")
```

Probabilité empirique de 4: 0.1

```
cat("Probabilité théorique:      ", round(1/6, 4), "\n")
```

Probabilité théorique: 0.1667

```
cat("Différence:                  ", round(p_empirique - 1/6, 4), "\n")
```

Différence: -0.0667

1.2.3 Observation

Lorsque l'on répète le lancer de 20 dés, la proportion de 4 **varie beaucoup** d'un échantillon à l'autre. Ceci est normal : avec un petit échantillon, l'estimateur a une **grande variabilité**.

Mais avec un nombre très grand de lancers (), la fluctuation devient **beaucoup plus faible**. C'est une illustration de la **Loi des Grands Nombres**.

```
# === Comparaison: petit vs. grand échantillon ===
```

```
cat("\n=== Effet de la Taille d'Échantillon ===\n\n")
```

```
=== Effet de la Taille d'Échantillon ===
```

```
# Petit échantillon: n = 100
n_petit <- 100
```

```

set.seed(11)
res_petit1 <- lancer_le_de(n_petit)
prop_petit1 <- sum(res_petit1 == 4) / n_petit

res_petit2 <- lancer_le_de(n_petit)
prop_petit2 <- sum(res_petit2 == 4) / n_petit

cat("Petit échantillon (n=100):\n")

```

Petit échantillon (n=100):

```
cat(" Tirage 1:  $\hat{p}$  =", round(prop_petit1, 4), "\n")
```

Tirage 1: $\hat{p}$ = 0.17

```
cat(" Tirage 2:  $\hat{p}$  =", round(prop_petit2, 4), "\n")
```

Tirage 2: $\hat{p}$ = 0.19

```
cat(" Différence:", round(abs(prop_petit1 - prop_petit2), 4), "\n\n")
```

Différence: 0.02

```

# Grand échantillon: n = 10000
n_grand <- 10000
set.seed(12)
res_grand1 <- lancer_le_de(n_grand)
prop_grand1 <- sum(res_grand1 == 4) / n_grand

res_grand2 <- lancer_le_de(n_grand)
prop_grand2 <- sum(res_grand2 == 4) / n_grand

cat("Grand échantillon (n=10000):\n")

```

Grand échantillon (n=10000):

```
cat(" Tirage 1:  $\hat{p}$  =", round(prop_grand1, 4), "\n")
```

Tirage 1: $\hat{p}$ = 0.1672

```
cat(" Tirage 2:  $\hat{p}$  =", round(prop_grand2, 4), "\n")
```

Tirage 2: $\hat{p}$ = 0.1718

```
cat(" Différence:", round(abs(prop_grand1 - prop_grand2), 4), "\n\n")
```

Différence: 0.0046

```
cat("Conclusion: Avec n grand, les deux estimations se rapprochent de p = 1/6 0.1667\n")
```

Conclusion: Avec n grand, les deux estimations se rapprochent de $p = 1/6$ 0.1667

1.2.4 Étape 2 : Estimation du Biais et de la Variance par Monte-Carlo

Objectif : Vérifier empiriquement que est un estimateur **sans biais** et calculer sa **variance**.

Plan :

1. Répéter M fois : simuler n lancers et calculer
2. Calculer la moyenne de ces M estimations (donne le biais)
3. Calculer la variance de ces M estimations

4. Comparer avec les formules théoriques

Formules théoriques :

- Biais de : (estimateur sans biais)
- Variance de :

```
# ===== Monte-Carlo: Estimer Biais et Variance =====
```

```
cat("\n=== Simulation Monte-Carlo pour  $\hat{p}$  ===\n\n")
```

```
=== Simulation Monte-Carlo pour  $\hat{p}$  ===
```

```
# Paramètres
n_obs <- 20 # Taille de chaque échantillon
M <- 10000 # Nombre de simulations

# Répéter M fois : simuler n lancers et calculer  $\hat{p}$ 
set.seed(123)
vecteur_p_hat <- replicate(M, {
  lancer <- lancer_le_de(n_obs)
  sum(lancer == 4) / n_obs
})

# Estimations empiriques
p_theo <- 1/6
biais_empirique <- mean(vecteur_p_hat) - p_theo
variance_empirique <- var(vecteur_p_hat)

cat("Paramètres de la simulation:\n")
```

Paramètres de la simulation:

```
cat("  n = taille de chaque échantillon:", n_obs, "\n")
```

n = taille de chaque échantillon: 20

```
cat("  M = nombre de répétitions:      ", M, "\n")
```

M = nombre de répétitions: 10000

```
cat("  p = probabilité théorique:      ", round(p_theo, 4), "\n\n")
```

p = probabilité théorique: 0.1667

```
cat("Estimations empiriques:\n")
```

Estimations empiriques:

```
cat("  E[ $\hat{p}$ ] (moyenne des M estimations):", round(mean(vecteur_p_hat), 4), "\n")
```

E[$\hat{p}$] (moyenne des M estimations): 0.1671

```
cat("  Biais = E[ $\hat{p}$ ] - p =                ", round(biais_empirique, 4), "\n")
```

Biais = E[$\hat{p}$] - p = 4e-04

```
cat("  Var( $\hat{p}$ ) (empirique) =                ", round(variance_empirique, 4), "\n\n")
```

Var($\hat{p}$) (empirique) = 0.0069

```
# Variance théorique
variance_theo <- p_theo * (1 - p_theo) / n_obs

cat("Valeurs théoriques:\n")
```

Valeurs théoriques:

```
cat("  Biais théorique:    0 (estimateur sans biais)\n")
```

Biais théorique: 0 (estimateur sans biais)

```
cat("  Var(p̂) théorique = p(1-p)/n = ", round(variance_theo, 4), "\n\n")
```

Var(p̂) théorique = p(1-p)/n = 0.0069

```
cat("Comparaison:\n")
```

Comparaison:

```
cat("  Erreur relative Var:", round(abs(variance_empirique - variance_theo)/variance_theo * 100, 2), "%")
```

Erreur relative Var: 0.01 %

Interprétation :

- L'estimateur est **sans biais** : (l'écart vient de la variabilité de simulation)
- La variance empirique est très proche de la formule théorique
- Ceci valide nos résultats théoriques!

1.2.5 Étape 3 : La Méthode du Bootstrap

Motivation : Dans la pratique, nous n'avons qu'un **seul échantillon**, pas des centaines de simulations. Comment estimer le biais et la variance avec un seul échantillon ?

Solution : Le Bootstrap

L'idée est de **rééchantillonner avec remise** à partir de l'échantillon observé pour générer de "pseudo-réplifications".

Algorithme :

1. Partir d'un échantillon observé
2. Rééchantillonner avec remise B fois pour générer B échantillons bootstrap
3. Calculer la statistique d'intérêt pour chacun
4. Estimer le biais et la variance des B estimations

```
# ===== BOOTSTRAP: Estimer Biais/Variance d'UN seul Échantillon =====

cat("\n=== Méthode du Bootstrap ===\n\n")
```

=== Méthode du Bootstrap ===

```
# Étape 1: Générer UN SEUL échantillon observé
n_obs <- 20
set.seed(456)
echantillon_obs <- lancer_le_de(n_obs)
p_hat_obs <- sum(echantillon_obs == 4) / n_obs

cat("Échantillon observé (n=20):\n")
```

Échantillon observé (n=20):

```
print(echantillon_obs)
```

```
[1] 5 5 3 6 5 4 3 1 6 2 1 2 3 3 6 6 5 5 4 6
```

```
cat("\nEstimateur observé:  $\hat{p}$  =", round(p_hat_obs, 4), "\n\n")
```

Estimateur observé: $\hat{p} = 0.1$

```
# Étape 2: Rééchantillonnage bootstrap
```

```
B <- 10000 # Nombre de rééchantillonnages bootstrap
```

```
vecteur_p_star <- replicate(B, {  
  # Rééchantillonner AVEC REMISE à partir de l'échantillon original  
  echantillon_resample <- sample(echantillon_obs, n_obs, replace = TRUE)  
  sum(echantillon_resample == 4) / n_obs  
})
```

```
# Étape 3: Estimer biais et variance bootstrap
```

```
biais_bootstrap <- mean(vecteur_p_star) - p_hat_obs
```

```
variance_bootstrap <- var(vecteur_p_star) * (B - 1) / B # Correction
```

```
cat("Résultats du Bootstrap (B=10000 rééchantillonnages):\n")
```

Résultats du Bootstrap (B=10000 rééchantillonnages):

```
cat("  Biais bootstrap = ", round(biais_bootstrap, 4), "\n")
```

```
Biais bootstrap = -3e-04
```

```
cat("  Variance bootstrap =", round(variance_bootstrap, 4), "\n\n")
```

```
Variance bootstrap = 0.0045
```

```
# Comparaison
```

```
cat("Comparaison avec résultats Monte-Carlo précédents:\n")
```

Comparaison avec résultats Monte-Carlo précédents:

```
cat("  Biais MC: ", round(biais_empirique, 4), "\n")
```

```
Biais MC: 4e-04
```

```
cat("  Variance MC: ", round(variance_empirique, 4), "\n\n")
```

```
Variance MC: 0.0069
```

```
cat("Conclusion: Le bootstrap donne des estimations proches des valeurs théoriques\n")
```

Conclusion: Le bootstrap donne des estimations proches des valeurs théoriques

```
cat("          à partir d'un SEUL échantillon observé.\n")
```

à partir d'un SEUL échantillon observé.

1.3 3.7.2 Simulation de Variables Aléatoires

1.3.1 Contexte

R dispose de nombreuses fonctions pour générer des variables aléatoires : `rnorm()`, `rbinom()`, `runif()`, etc.

Mais que faire si la loi n'existe pas dans R ?

Nous présentons deux **méthodes classiques** :

1. **Transformation inverse** : utiliser l'inverse de la fonction de répartition
2. **Simulation discrète** : diviser l'intervalle $[0,1]$ en sous-intervalles

1.3.2 Méthode 1 : Transformation Inverse pour Loi Exponentielle

Théorie : Si F est la fonction de répartition, et U , alors :

Pour la loi exponentielle :

Inverse :

Avantage : Méthode simple et efficace!

On sait que si alors sa fonction de répartition est

L'inverse de cette fonction est :

On peut donc simuler observations de en utilisant des réalisations issues d'une loi uniforme sur .

```
# ===== Transformation Inverse: Loi Exponentielle =====
```

```
cat("\n=== Simulation de Exp(=2) par Transformation Inverse ===\n\n")
```

```
=== Simulation de Exp(=2) par Transformation Inverse ===
```

```
# Paramètres
```

```
n <- 10000
```

```
lambda <- 2
```

```
# Étape 1: Générer  $U \sim \text{Uniform}(0,1)$ 
```

```
U <- runif(n)
```

```
# Étape 2: Appliquer la formule inverse
```

```
X_exp <- -log(1 - U) / lambda
```

```
cat("Données simulées: n =", n, "observations d'Exp(2)\n\n")
```

```
Données simulées: n = 10000 observations d'Exp(2)
```

```
cat("Vérification statistique:\n")
```

```
Vérification statistique:
```

```
cat(" Moyenne observée: ", round(mean(X_exp), 4), "\n")
```

```
Moyenne observée: 0.4986
```

```
cat(" Moyenne théorique: ", 1/lambda, "(car  $E[\text{Exp}()]=1/$ )\n")
```

```
Moyenne théorique: 0.5 (car  $E[\text{Exp}()]=1/$ )
```

```
cat(" Variance observée: ", round(var(X_exp), 4), "\n")
```

Variance observée: 0.2468

```
cat(" Variance théorique: ", 1/lambda^2, "(car Var[Exp()]=1/ ^2)\n\n")
```

Variance théorique: 0.25 (car $\text{Var}[\text{Exp}()]=1/2^2$)

```
# Visualisation
```

```
hist(X_exp, breaks = 40, freq = FALSE, col = "lightgreen",
     main = "Simulation Exp(=2) par Transformation Inverse",
     xlab = "Valeur",
     ylab = "Densité")
```

```
# Superposer la densité théorique
```

```
curve(dexp(x, rate = lambda), add = TRUE, col = "red", lwd = 2)
```

```
legend("topright",
```

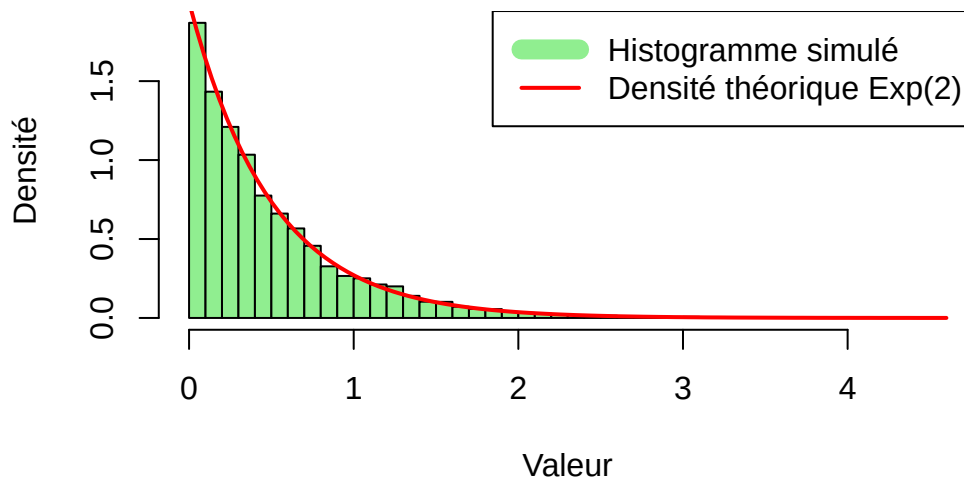
```
     legend = c("Histogramme simulé", "Densité théorique Exp(2)"),
```

```
     col = c("lightgreen", "red"),
```

```
     lty = c(1, 1),
```

```
     lwd = c(10, 2))
```

Simulation $\text{Exp}(=2)$ par Transformation Inverse



```
cat("Les deux courbes se superposent: la simulation est valide !\n")
```

Les deux courbes se superposent: la simulation est valide !

Les deux courbes se superposent, confirmant que la simulation correspond bien à une loi exponentielle de paramètre 2.

1.3.3 Méthode 2 : Simulation Discrète Uniforme

Objectif : Simuler une variable aléatoire **discrète** uniforme sur .

Principe :

- Diviser l'intervalle en 6 sous-intervalles égaux :
- Si , alors
- Mathématiquement : où est la partie entière

Avantage : Fonctionne pour n'importe quelle distribution discrète!

```
# ===== Simulation Discrète: Uniforme sur {0,1,2,3,4,5} =====
```

```
cat("\n=== Simulation Discrète Uniforme ===\n\n")
```

```
=== Simulation Discrète Uniforme ===
```

```
# Paramètres
```

```
n <- 10000
```

```
# Étape 1: Générer  $U \sim \text{Uniform}(0,1)$ 
```

```
U <- runif(n)
```

```
# Étape 2: Transformer en variable discrète
```

```
# La formule  $\text{floor}(U * 6)$  produit un entier dans  $\{0,1,2,3,4,5\}$ 
```

```
X_discrete <- floor(U * 6)
```

```
cat("Nombre d'observations: n =", n, "\n\n")
```

```
Nombre d'observations: n = 10000
```

```
# Compter les fréquences
```

```
freq <- table(X_discrete)
```

```
cat("Fréquences observées:\n")
```

```
Fréquences observées:
```

```
print(freq)
```

```
X_discrete
```

```
  0    1    2    3    4    5  
1663 1649 1658 1720 1676 1634
```

```
cat("\nFréquences théoriques (1/6 chacun ", round(n/6, 0), ")\n\n")
```

```
Fréquences théoriques (1/6 chacun 1667 )
```

```
# Visualisation
```

```
barplot(freq, col = "orange",  
        main = "Simulation Discrète Uniforme sur {0,1,2,3,4,5}",  
        xlab = "Valeur",  
        ylab = "Fréquence",  
        ylim = c(0, max(freq) * 1.2))
```

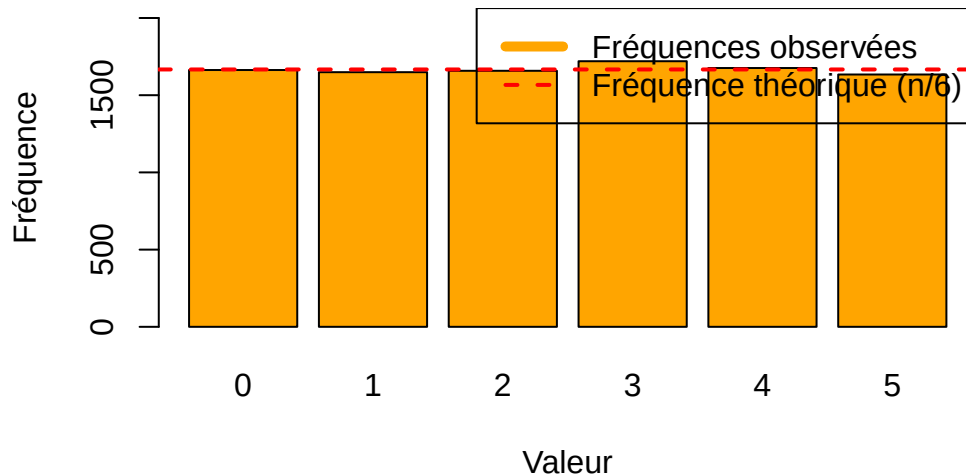
```
# Ajouter une ligne référence pour l'équilibre
```

```
abline(h = n/6, col = "red", lwd = 2, lty = 2)
```

```
legend("topright",
```

```
      legend = c("Fréquences observées", "Fréquence théorique (n/6)",  
      col = c("orange", "red"),  
      lty = c(1, 2),  
      lwd = c(5, 2))
```

Simulation Discrète Uniforme sur {0,1,2,3,4,5}



```
cat("Interprétation: Les barres sont de hauteur égale (légères variations normales)\n")
```

Interprétation: Les barres sont de hauteur égale (légères variations normales)

```
cat("Ceci confirme que chaque valeur a probabilité 1/6\n")
```

Ceci confirme que chaque valeur a probabilité 1/6

1.4 Résumé et Points Clés

1.4.1 Ce que nous avons appris

1. Bootstrap :

- Méthode puissante pour estimer biais et variance à partir d'un **seul échantillon**
- Rééchantillonnage avec remise pour générer des pseudo-répliques
- Utile quand les formules théoriques ne sont pas disponibles

2. Transformation Inverse :

- Utiliser l'inverse de la fonction de répartition
- Exemple : où
- Efficace et simple à implémenter

3. Simulation Discrète :

- Diviser $[0,1]$ en sous-intervalles selon les probabilités
- Appliquer une fonction comme
- Généralise à n'importe quelle distribution discrète

1.4.2 Conseils Pratiques

- Toujours **vérifier la simulation** visuellement (histogramme) et numériquement (statistiques)
- Utiliser `set.seed()` pour **reproduire les résultats**
- Adapter le nombre de répliques (ou) selon la **précision souhaitée**
- Le bootstrap est **non-paramétrique** : ne suppose rien sur la loi sous-jacente